

Qudit-ADAPT para Multiway Number Partitioning

Informe de avance

28 de septiembre de 2026

Resumen

Extendemos Qudit-ADAPT al problema de partición de números en tres subconjuntos (*multiway number partitioning*, $k = 3$), en el que cada número se codifica en un qutrit y su nivel es la caja a la que se asigna. Construimos un motor de simulación que evita toda matriz densa y entrega gradientes analíticos al optimizador, lo validamos contra la implementación de J. Molina reconstruyendo sus 40 *ansätze* con diferencia máxima de energía 7.7×10^{-12} , y lo usamos para correr 478 simulaciones en BitWit sobre instancias de $n = 5$ a 10 qutrits con solución única. El pool de orden $\ell = 2$ resuelve muchas más instancias que el de $\ell = 1$, pero la tasa de éxito cae con n para ambos. La comparación entre inicializar desde el óptimo anterior (*warm start*) y desde $\theta = 0$ no arroja un efecto consistente con la estadística actual. El hallazgo principal es metodológico: el resultado de una corrida individual depende del orden, arbitrario, en que se asignan los números a los qutrits, porque los empates en la selección de operadores se resuelven por índice. Promediar sobre órdenes cambia la tasa de éxito medida de 50 % a 66 % en $n = 6$, y obliga a rediseñar el protocolo antes de ampliar la estadística.

Contenido

1	Introducción	2
2	Metodología	2
2.1	Formulación	2
2.2	Pool de operadores	2
2.3	Motor de simulación	3
2.4	Instancias	3
2.5	Protocolo y estrategias de inicialización	3
2.6	Métricas	4
2.7	Validación y ejecución	4
3	Resultados	4
3.1	Validación contra la implementación de J. Molina	4
3.2	Fase 1: $\ell = 1$ contra $\ell = 2$	4
3.3	Respuestas perdidas	4
3.4	Fase 2: <i>warm start</i> contra $\theta = 0$	5
3.5	¿Importa el rango de los números?	5
3.6	Sensibilidad al orden de los números y a la máquina	6
3.7	Varianza del gradiente	7
3.8	Una instancia de $n = 13$	7
3.9	Costo computacional	7
4	Análisis	8
5	Limitaciones y próximos pasos	9

1 Introducción

Qudit-ADAPT es una versión para qudits de ADAPT-VQE en la que el ansatz se construye de a un operador por vez, eligiendo en cada paso el de mayor gradiente dentro de un pool derivado de un potencial de gauge adiabático aproximado. En el manuscrito se aplicó a Max 3-Cut. Aquí lo extendemos a un segundo problema de optimización ternaria: repartir n números positivos en tres cajas de modo que sus sumas queden lo más parejas posible.

El problema es natural para qutrits. Con qubits, una etiqueta de tres valores requiere codificación *one-hot* y un término de penalización que prohíba los estados inválidos; con un qutrit por número, cada estado de la base es una asignación válida y el nivel del qutrit *es* la caja.

J. Molina implementó una primera versión y la corrió para $n = 5$ y 6 . Este informe describe una reimplementación orientada a llegar a tamaños mayores, un benchmark sistemático hasta $n = 10$, y los resultados obtenidos hasta ahora. Los objetivos son tres:

1. comparar las dos truncaciones del pool, $\ell = 1$ y $\ell = 2$, a lo largo de n ;
2. poner a prueba la conjetura de que, en este problema, partir del óptimo anterior (*warm start*) es una desventaja frente a partir de la superposición uniforme;
3. contar el costo en parámetros y en compuertas nativas.

2 Metodología

2.1 Formulación

Sean $a_1, \dots, a_n$ los números y $z_j \in \{0, 1, 2\}$ la caja asignada al número j . Seguimos la formulación de J. Molina (`formulacion_qubo.pdf`): con $\Sigma_i(z) = \sum_j a_j \delta_{i,z_j}$ la suma de la caja i y $\mu = \frac{1}{3} \sum_j a_j$,

$$C(z) = \sum_{i=0}^2 [\Sigma_i(z) - \mu]^2, \quad (1)$$

que vale exactamente cero en una partición perfecta. La etiqueta de cada qutrit es el autovalor del operador dígito $\hat{d} = \hat{J}_z + \hat{I}$; como $\hat{J}_z = \text{diag}(1, 0, -1)$, el nivel m del qutrit lleva la etiqueta $2 - m$. Adoptamos esta convención para que las cadenas de trits se lean igual que en su trabajo.

El Hamiltoniano de costo coincide, salvo una transformación afín, con el de Max 3-Cut sobre el grafo completo con pesos $a_i a_j$:

$$H_C = 2 H_{3\text{-cut}}^{(a_i a_j)} + \frac{2}{3} \left(\sum_j a_j \right)^2, \quad (2)$$

lo que verificamos numéricamente con residuo nulo. Dos consecuencias importan. Como todos los pares se acoplan, el pool de una instancia de n números es el pool del grafo completo K_n , que es el caso más denso. Y el error relativo $|E - E_0|/|E_0|$ no sirve aquí: en instancias balanceadas $E_0 = 0$ y el cociente se indefine. Reportamos por eso energías en la escala de la ec. (1), donde E es la desviación cuadrática media de las sumas de las cajas.

2.2 Pool de operadores

El pool se obtiene de los conmutadores anidados del Hamiltoniano adiabático, truncados a orden ℓ : con $\ell = 1$ se usa O_1 y con $\ell = 2$ se agrega O_3 . Sobre K_n , el pool de $\ell = 1$ tiene exactamente $2n^2 - n$ operadores (190 en $n = 10$) y el de $\ell = 2$ crece como $n^{3.65}$ (18 100 en $n = 10$). Cada operador actúa sobre a lo más cuatro qutrits.

2.3 Motor de simulación

El código original de la línea Max 3-Cut no escala más allá de $n \approx 8$, por dos razones evitables. Construía H_C como matriz densa de $(3^n)^2$ entradas y la diagonalizaba (52 GB en $n = 10$), y guardaba en forma densa la base propia de cada operador del ansatz (1.4 GB por operador en $n = 8$). Ninguna de las dos hace falta:

- H_C es diagonal en la base computacional, así que se guarda como un vector de 3^n entradas y E_0 es su mínimo; el estado de referencia es el producto conocido $|+_3\rangle^{\otimes n}$.
- Cada operador del pool actúa en a lo más cuatro sitios. Su bloque de $3^w \times 3^w$ (81×81 como máximo) se diagonaliza una vez, y $e^{-i\theta A}$ se aplica contrayendo ese bloque contra el estado. Es exacto incluso para los operadores cuya hermitización no factoriza como producto tensorial.
- El gradiente es analítico, por el método adjunto: una pasada hacia adelante y una hacia atrás, con costo del orden de dos evaluaciones de la energía. Se entrega a BFGS junto con la función de costo (jac=True). Verificado contra diferencias finitas a 10^{-8} .
- La expansión simbólica del pool se hace sólo hasta el orden que ℓ requiere y se guarda en disco. Los pools de $n = 6$ a 14 con $\ell = 1$ se construyen en menos de dos segundos, y el de $n = 10$ con $\ell = 2$ en 3.6 s.

2.4 Instancias

Cada instancia cumple tres criterios, verificados por enumeración exacta: números *distintos*; existencia de un reparto perfecto; y unicidad de ese reparto, es decir, degeneración del fundamental exactamente igual a 6, el mínimo posible porque cada partición aparece una vez por cada permutación de las tres etiquetas. Los números repetidos se excluyen porque inflan la degeneración y con ella la probabilidad de acertar por azar. Con números distintos no existe reparto perfecto para $n < 5$, por lo que el benchmark cubre $n = 5, \dots, 10$.

Los números se sortean en $[1, 3n]$. El rango crece con n porque, fijo en $[1, 15]$ como en las instancias de J. Molina, en $n = 10$ no queda ninguna instancia de solución única. Se fijaron desde el comienzo 20 instancias por tamaño, de forma determinista, aunque la primera tanda use sólo las 10 primeras; así la segunda no se elige a posteriori. La tabla 1 muestra una por tamaño junto con su solución.

Tabla 1: Una instancia de cada tamaño y su única partición perfecta. $p_{\text{éxito}}^{\text{azar}} = 6/3^n$ es la probabilidad de acertar midiendo la superposición uniforme.

n	rango	$p_{\text{éxito}}^{\text{azar}}$	números	cajas óptimas
5	[1, 15]	2.5×10^{-2}	[2, 3, 7, 8, 10]	10 3, 7 2, 8
6	[1, 18]	8.2×10^{-3}	[1, 6, 8, 10, 12, 17]	8, 10 6, 12 1, 17
7	[1, 21]	2.7×10^{-3}	[1, 4, 6, 10, 13, 16, 19]	10, 13 4, 19 1, 6, 16
8	[1, 24]	9.1×10^{-4}	[8, 9, 10, 11, 12, 14, 16, 19]	14, 19 10, 11, 12 8, 9, 16
9	[1, 27]	3.0×10^{-4}	[1, 2, 5, 6, 7, 8, 20, 21, 26]	6, 26 5, 7, 20 1, 2, 8, 21
10	[1, 30]	1.0×10^{-4}	[2, 3, 12, 15, 16, 17, 18, 22, 27, 30]	12, 15, 27 3, 16, 17, 18 2, 22, 30

2.5 Protocolo y estrategias de inicialización

ADAPT corre hasta que la norma del gradiente del pool cae bajo $\epsilon = 10^{-3}$ (en la escala de la ec. 1) o hasta 150 parámetros. El optimizador interno es BFGS con $\text{gtol} = 10^{-10}$. Se comparan tres estrategias:

warm start. El ADAPT estándar: cada optimización parte de $(\theta_{k-1}^*, 0)$, el óptimo anterior más el parámetro nuevo en cero.

(i) **secuencia fija desde $\theta = 0$.** Se toma la secuencia de operadores que eligió la corrida warm y, para cada tamaño k , se reoptimiza ese mismo circuito partiendo de cero. Aísla el efecto de la inicialización sobre un circuito dado; es el experimento de las Figs. 5–6 del manuscrito.

- (ii) **ADAPT desde $\theta = 0$.** En cada paso se reoptimizan todos los parámetros partiendo de cero, es decir, desde la superposición uniforme. Como el estado optimizado cambia, el barrido siguiente elige otros operadores: es otro algoritmo, y es la variante que pone a prueba la conjetura.

2.6 Métricas

La métrica principal es $p_{\text{éxito}}$, el peso del estado final sobre el subespacio de las soluciones óptimas: la probabilidad de que una medición entregue la partición correcta. Consideramos una instancia *resuelta* si $p_{\text{éxito}} \geq 0.1$; con esa probabilidad, 50 mediciones bastan para ver la solución con más de 99 % de certeza, y verificar cuál de ellas es óptima es trivial clásicamente. Un criterio más estricto, pedir que la cadena más probable sea la correcta, cuenta como fallas estados con la mitad de su peso sobre la solución, y lo descartamos.

Se registran además, en cada iteración, la energía, la norma del gradiente, $p_{\text{éxito}}$, los parámetros, los empates en la selección y los tiempos; el número de parámetros, las compuertas nativas según el Algoritmo 1 de Ringbauer *et al.* y el commit exacto del código que produjo cada resultado.

2.7 Validación y ejecución

El código de J. Molina no corre desde un clon del repositorio porque depende de un módulo que no está publicado, pero sus resultados guardan los operadores y parámetros finales de cada corrida. Reconstruimos sus 40 ansätze en nuestro motor y comparamos la energía del estado resultante con la suya.

Todas las corridas del benchmark se hicieron en BitWit (AMD Threadripper 7980X, 64 núcleos, 125 GB), en paralelo, con varios procesos de pocos hilos de BLAS cada uno, a partir de árboles exportados de commits fijos del repositorio.

3 Resultados

3.1 Validación contra la implementación de J. Molina

Los 40 ansätze reconstruidos dan su energía con diferencia máxima 7.7×10^{-12} , y la probabilidad de la cadena más probable coincide en los 40. Las dos implementaciones concuerdan en el estado de referencia, la hermitización del pool, las convenciones de signo y de orden, y el Hamiltoniano.

Corriendo además nuestro ADAPT sobre sus 40 instancias con sus mismos ajustes ($\epsilon = 10^{-3}$, tope de 30), $p_{\text{éxito}}$ coincide dentro de 0.05 en 35 de 40 casos; en 4 el nuestro llega más alto y en 1 el suyo. Las diferencias vienen de que su pool contiene n operadores de un sitio más: sus proyectores son cuadráticos en $\hat{d}$ y la expansión simbólica produce más monomios distintos.

3.2 Fase 1: $\ell = 1$ contra $\ell = 2$

La tabla 2 y la figura 1 resumen las 120 corridas con warm start. El pool de $\ell = 2$ resuelve muchas más instancias: sobre la misma instancia, $\ell = 2$ acierta y $\ell = 1$ no en 17 casos, y lo contrario ocurre en 3. Para ambos pools la tasa de éxito cae con n , y en $n = 10$ casi ninguna instancia se resuelve.

Dos salvedades. Con $\ell = 1$, 58 de las 60 corridas convergieron por el criterio de gradiente: sus fallas son mínimos locales genuinos, no falta de presupuesto. Con $\ell = 2$, en cambio, muchas corridas de $n \geq 7$ llegaron al tope de 150 parámetros (tabla 2), así que sus resultados ahí son cotas inferiores.

3.3 Respuestas perdidas

En 11 de las 41 corridas con $\ell = 1$ que no resuelven la instancia, $p_{\text{éxito}}$ llegó a mitad de camino a más de diez veces el azar antes de caer. La figura 2 muestra dos casos. En ambos el warm start sube $p_{\text{éxito}}$ por encima del 10 % y después la lleva a menos de 10^{-12} mientras la energía sigue bajando. Minimizar $\langle H_C \rangle$ no exige concentrarse en la solución: basta con repartir el peso sobre configuraciones cercanas, y el fondo del espectro de este problema es denso.

Tabla 2: Fase 1, warm start, 10 instancias por tamaño. “Conv.” cuenta las corridas que terminaron por el criterio de gradiente y no por el tope de 150 parámetros; “nativas” y “MS” son medianas del total de compuertas nativas y de compuertas Mølmer–Sørensen.

n	ℓ	resueltas	$p_{\text{éxito}}$ mediana	k med.	conv.	nativas	MS	tiempo
5	1	7/10	9.7×10^{-1}	16	10/10	204	28	0.4 s
	2	9/10	1.0×10^0	16	10/10	272	46	1.4 s
6	1	2/10	9.6×10^{-18}	20	10/10	246	34	0.8 s
	2	9/10	1.0×10^0	31	10/10	559	97	6.6 s
7	1	5/10	5.1×10^{-2}	30	10/10	386	54	2.2 s
	2	7/10	5.0×10^{-1}	60	7/10	1130	204	49.5 s
8	1	2/10	2.3×10^{-13}	22	10/10	286	40	2.6 s
	2	3/10	3.5×10^{-7}	138	5/10	2630	465	16.7 min
9	1	3/10	4.5×10^{-3}	38	8/10	494	69	1.9 min
	2	3/10	1.1×10^{-16}	64	8/10	1195	217	19.1 min
10	1	0/10	4.9×10^{-36}	54	10/10	710	100	10.2 min
	2	2/10	6.4×10^{-14}	150	3/10	2860	527	2.7 h

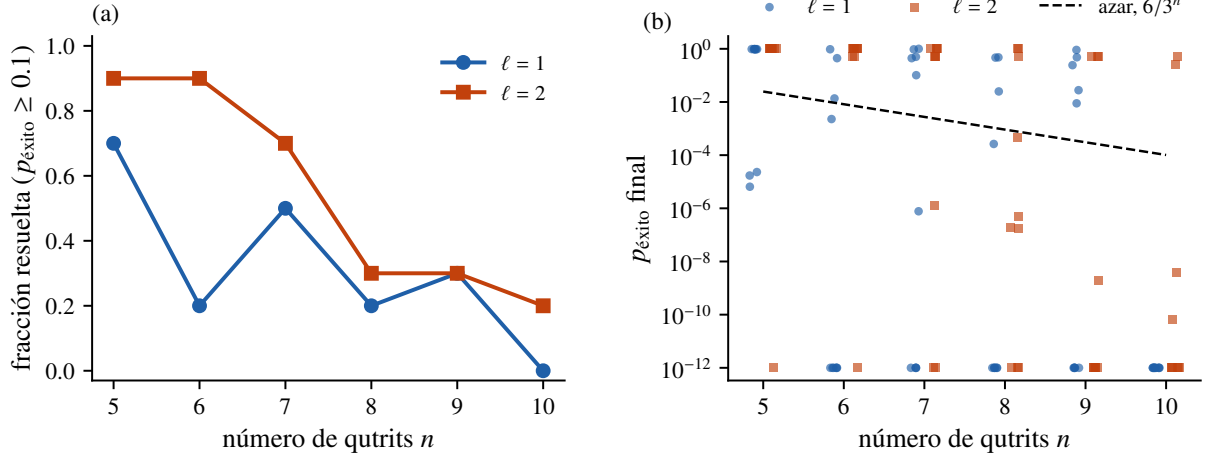


Figura 1: (a) Fracción de instancias resueltas ($p_{\text{éxito}} \geq 0.1$) en función de n . (b) $p_{\text{éxito}}$ final de cada instancia; la línea discontinua es el azar, $6/3^n$. Las probabilidades menores que 10^{-12} se grafican sobre el piso.

3.4 Fase 2: warm start contra $\theta = 0$

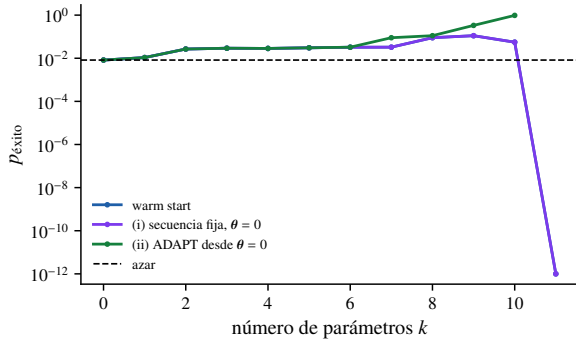
La tabla 3 y la figura 3 comparan las tres estrategias con $\ell = 1$ sobre las mismas instancias. Contamos que una estrategia “gana” cuando su $p_{\text{éxito}}$ es más del doble de la otra y mayor que 10^{-6} .

ADAPT desde $\theta = 0$ gana en 12 instancias y el warm start en 8, pero el signo depende de n : en $n = 6$ gana (ii) por seis a uno y en $n = 7$ gana warm por tres a cero. Con la secuencia fija, el warm start es igual o mejor (5 a 3), lo que es coherente con el argumento de *burrowing*: sobre un circuito dado, partir del óptimo anterior garantiza un descenso monótono.

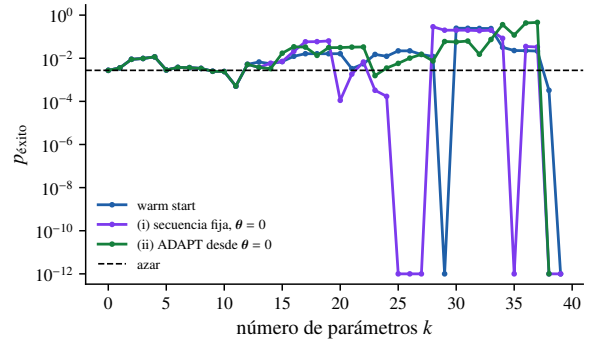
3.5 ¿Importa el rango de los números?

Nuestro motor resuelve las cuatro instancias sin repetidos de $n = 6$ de J. Molina, pero sólo 2 de nuestras 10. Como sus números llegan hasta 8 y los nuestros hasta 18, probamos si la magnitud explica la diferencia. Con $n = 6$, números distintos y solución única existen exactamente 9 instancias en $[1, 8]$; las corrimos todas, junto con las 10 instancias restantes de la segunda tanda en $[1, 18]$ (figura 4).

El rango no explica nada: con $\ell = 1$ se resuelven 3/9 en $[1, 8]$ y 10/20 en $[1, 18]$, y con $\ell = 2$, 8/9 y



(a) $n = 6$, instancia 3.



(b) $n = 7$, instancia 4.

Figura 2: $p_{\text{éxito}}$ en función del número de parámetros para las tres estrategias. En (a), el warm start alcanza $p_{\text{éxito}} = 0.11$ y la pierde, la misma secuencia reoptimizada desde cero también falla, y ADAPT desde cero acierta con $p_{\text{éxito}} = 0.97$. En (b) ninguna estrategia la recupera.

Tabla 3: Fase 2 con $\ell = 1$. Las tres primeras columnas son instancias resueltas de 10; las dos últimas, comparaciones instancia a instancia en la forma “gana warm / gana la otra / empate”.

n	resueltas			contra warm	
	warm	(ii)	(i)	(ii) ADAPT desde 0	(i) secuencia fija
5	7	6	6	1 / 3 / 6	1 / 0 / 9
6	2	8	3	1 / 6 / 3	0 / 2 / 8
7	5	3	3	3 / 0 / 7	2 / 0 / 8
8	2	2	2	1 / 1 / 8	0 / 0 / 10
9	3	4	3	2 / 2 / 6	2 / 1 / 7
10	0	0	0	0 / 0 / 10	0 / 0 / 10

18/20. Además la segunda tanda de $[1, 18]$ resolvió 8 de 10, contra 2 de la primera: el resultado de la fase 1 en $n = 6$ era una muestra desfavorable, no una propiedad de las instancias.

3.6 Sensibilidad al orden de los números y a la máquina

La diferencia con J. Molina resultó venir de otro lado. Nuestro benchmark ordena los números de menor a mayor antes de asignarlos a los qutrits; él no. Con los mismos números en su orden y $\ell = 1$, las dos instancias de $[1, 8]$ que coinciden con las suyas se resuelven con $p_{\text{éxito}} = 0.49$ y 0.97 ; ordenadas, con $p_{\text{éxito}} = 0$.

Permutar qué número va en qué qutrit no cambia el problema, pero sí puede cambiar la trayectoria de ADAPT. Cuando varios operadores empatan en el gradiente, argmax toma el de menor índice, y el índice depende del etiquetado; un solo empate basta para que dos trayectorias diverjan. Para cuantificarlo corrimos las 20 instancias de $n = 6$ bajo 10 órdenes cada una (figura 5). 9 se resuelven en cualquier orden, 3 en ninguno y 8 dependen del orden. Con el orden ascendente se resuelve el 50%; promediando sobre órdenes, el 66%.

El orden ascendente parece además sistemáticamente desfavorable: de las 8 instancias que dependen del orden, falla en 7. Una hipótesis, todavía sin probar, es que al desempatar por índice menor con los números en orden creciente se favorecen operadores sobre los números más chicos, que son los que menos pesan en el balance.

El mismo fenómeno aparece con el hardware. Repetimos en una laptop las 20 corridas de $n = 5$ y 6 con $\ell = 1$: con uno o dos hilos de BLAS el veredicto no cambia en ningún caso (0/20), pero entre la laptop y BitWit cambia en 1 de 20. Distintos procesadores y builds de BLAS redondean distinto, y en un casi-empate esa diferencia de 10^{-16} basta para bifurcar la trayectoria. Todas las corridas del benchmark son de BitWit y son consistentes entre sí, pero una corrida individual no debe tomarse como propiedad robusta

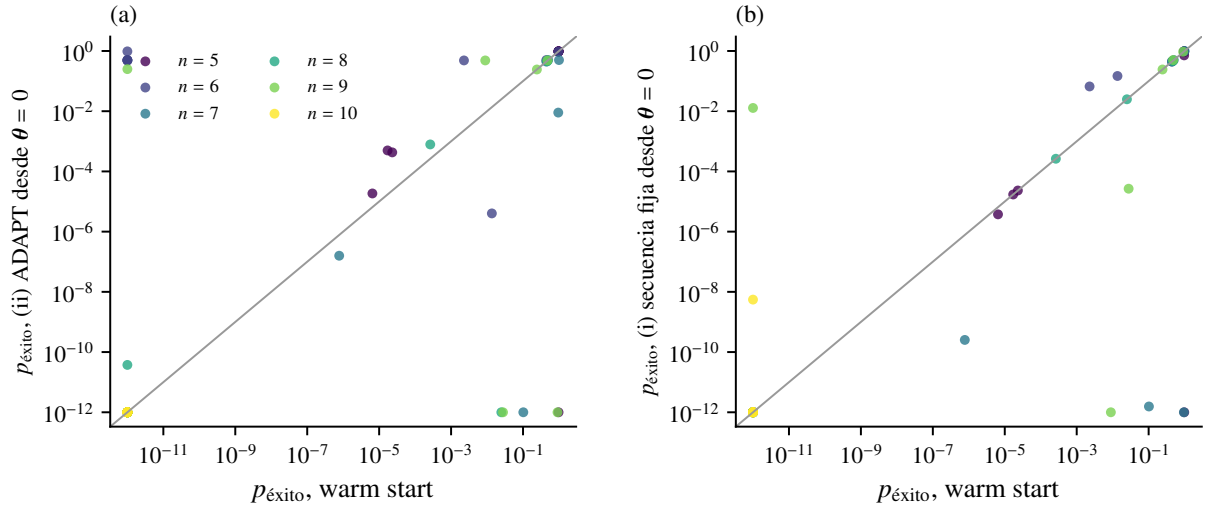


Figura 3: $p_{\text{éxito}}$ de cada instancia con warm start contra (a) ADAPT desde $\theta = 0$ y (b) la secuencia fija reoptimizada desde cero. Sobre la diagonal, gana la estrategia del eje vertical.

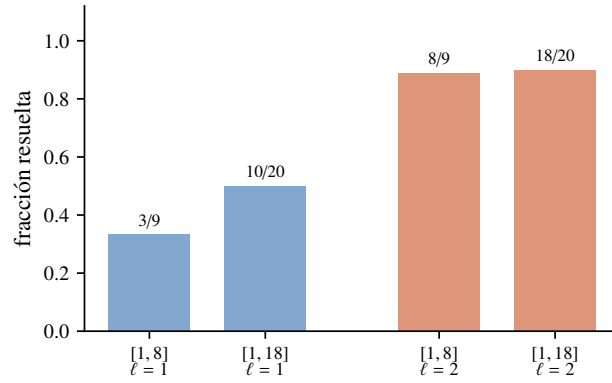


Figura 4: Fracción resuelta en $n = 6$ según el rango de los números.

de su instancia.

3.7 Varianza del gradiente

En un barrido anterior medimos la varianza del gradiente en puntos de parámetros aleatorios contra el número de qutrits, de $n = 6$ a 14, con ansätze de $k = 5, 10$ y 20 operadores sorteados del pool de $\ell = 1$ y H_C normalizado por su rango espectral (figura 6). Un ajuste exponencial da $\text{Var} \propto e^{-\alpha n}$ con $\alpha = 0.38$, contra $\ln 3 \approx 1.10$ de un ansatz que se acerque a un 2-diseño; en $n = 14$ la varianza medida es 241 veces mayor que la de esa referencia, y apenas cambia con la profundidad.

3.8 Una instancia de $n = 13$

Una corrida exploratoria con $n = 13$ y $\ell = 1$ convergió por gradiente en 64 parámetros tras 7.7 h. La probabilidad de medir la solución alcanzó su máximo, 2.9×10^{-3} , en $k = 12$, y terminó en 3.1×10^{-45} : menos que el azar. Es el mismo fenómeno de la sección de respuestas perdidas, llevado al extremo.

3.9 Costo computacional

Con $\ell = 1$ casi todo el tiempo se va en BFGS; con $\ell = 2$, entre un cuarto y casi la mitad se va en el barrido de gradiente del pool (figura 7). En $n = 10$ una corrida con $\ell = 2$ toma del orden de horas. El costo por

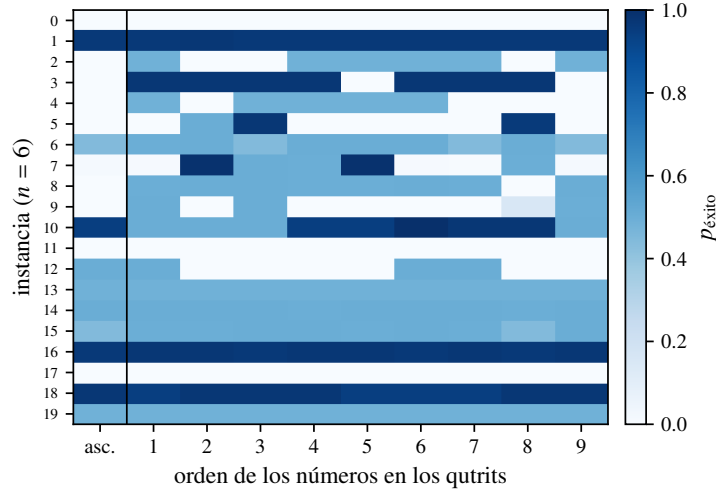


Figura 5: $p_{\text{éxito}}$ de las 20 instancias de $n = 6$ ($\ell = 1$, warm start) bajo 10 órdenes de sus números. La primera columna es el orden ascendente del benchmark.

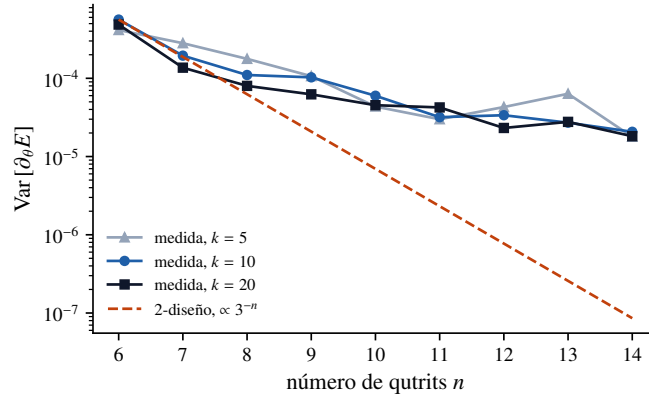


Figura 6: Varianza del gradiente en puntos aleatorios contra n , para tres profundidades, y la referencia de un 2-diseño anclada en $n = 6$.

evaluación de la energía crece entre $n = 8$ y 9 bastante más que el tamaño del estado, lo que atribuimos a contención de memoria entre los procesos concurrentes y a copias evitables del estado en la contracción de bloques; queda por medirlo aislado.

4 Análisis

El pool importa más que la inicialización. Pasar de $\ell = 1$ a $\ell = 2$ es, con diferencia, lo que más aumenta la tasa de éxito. El efecto de la estrategia de inicialización es, en comparación, pequeño y de signo variable. Esto es consistente con la imagen de que el cuello está en la expresividad del pool: $\ell = 1$ contiene sólo operadores de uno y dos sitios.

Energía y solución se desacoplan. Las respuestas perdidas y la corrida de $n = 13$ muestran que bajar la energía no equivale a acercarse a la solución. El error de energía, en particular el error relativo, es una métrica engañosa para este problema; la cifra relevante es $p_{\text{éxito}}$.

La conjetura sobre el warm start sigue abierta. Con 10 instancias por tamaño no hay un efecto consistente. Hay, sin embargo, una distinción que vale la pena mantener: sobre un circuito fijo el warm start no es peor que partir de cero, así que si la conjetura resulta cierta, el mecanismo tendría que estar en la

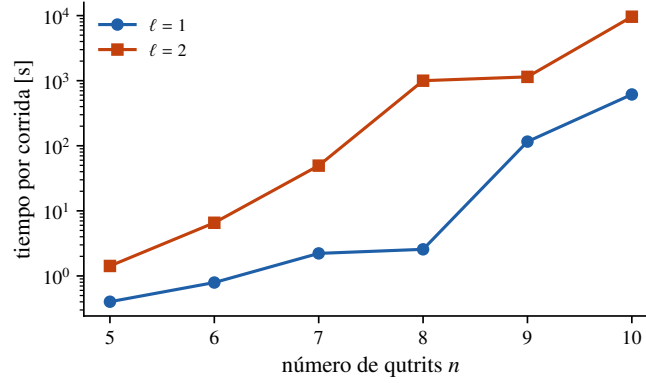


Figura 7: Tiempo mediano por corrida en BitWit, con warm start.

selección de operadores y no en la inicialización. Una observación de avances anteriores de este trabajo, un ejemplo en que la diferencia parecía nítida, resultó depender de la máquina en que se corrió y se descartó.

El protocolo tiene que promediar sobre órdenes. Éste es el resultado que más condiciona lo que sigue. Si 8 de 20 instancias cambian de veredicto según un orden arbitrario, una sola corrida por instancia mide tanto el orden como el algoritmo, y comparar dos estrategias sobre un orden fijo puede reflejar cuál de ellas tuvo un empate a favor. La tasa de éxito debe estimarse promediando sobre varios órdenes, o sobre un desempate aleatorio con semilla, y eso vale también para reevaluar la fase 2.

Barren plateaus. La varianza del gradiente cae con n mucho más lento que la de un 2-diseño y no depende de la profundidad, lo que va en la dirección de lo que el manuscrito afirma. Con nueve puntos entre $n = 6$ y 14, sin embargo, no se distingue un decaimiento exponencial lento de uno polinomial: sobre ese rango las dos curvas casi coinciden.

5 Limitaciones y próximos pasos

- **Promediar sobre órdenes.** Rehacer las fases 1 y 2 con varios órdenes por instancia, o con desempate aleatorio, antes de ampliar a 20 instancias por tamaño. Probar la hipótesis de que el orden ascendente es desfavorable.
- **Presupuesto de $\ell = 2$.** Subir el tope de parámetros para $n \geq 7$, donde muchas corridas no convergieron.
- **Rendimiento.** Medir aislado el costo por evaluación, eliminar las copias del estado en la contracción de bloques y ajustar el número de procesos concurrentes al ancho de banda de memoria.
- **Comparación con otros algoritmos.** Queda para el final, según lo acordado.
- **Código de J. Molina.** Su implementación depende de un módulo que no está en el repositorio; nuestro motor la reemplaza y está validado contra sus resultados.